

MARAN PMS — Diseño: Integración Inventario + Restaurant + Descuento Automático de Stock

Estado actual del sistema (diagnóstico pre-implementación)

Lo que existe y funciona

- `inventory_items` — artículos con SKU, categoría, proveedor, stock mínimo/máximo, `currentStock`
- `stock_movements` — registro de entradas/salidas/ajustes con `previousStock` y `newStock`
- `item_categories` — categorías por área (restaurant, spa, housekeeping, mantenimiento, admin)
- `recipes` — una receta por ítem del menú
- `recipe_ingredients` — ingredientes con cantidad, unidad y costo unitario
- La UI de recetas muestra costo vs precio y calcula margen

Bugs a corregir antes de la integración

Bug 1 — Stock no se actualiza en DB: El endpoint `POST /api/inventory/movements` calcula `newStock` correctamente pero solo guarda el movimiento en `stock_movements`. No actualiza `currentStock` en `inventory_items`. El in-memory storage sí lo hace.

Fix inmediato en `server/routes/inventory.ts`, endpoint `POST /api/inventory/movements`:

```
// Agregar DESPUÉS de createStockMovement:  
await storage.updateInventoryItem(itemId, { currentStock: newStock });
```

Bug 2 — Ingredientes de receta no vinculados al inventario:

`recipe_ingredients.inventory_item_id` existe en el schema pero la UI carga ingredientes como texto libre sin asociarlos a un ítem real de `inventory_items`.

Arquitectura de la integración

Flujo objetivo

```
Carga de inventario (entrada)
  → inventory_items.currentStock aumenta
  → stock_movements registra entrada

Venta en restaurant (ticket cerrado)
  → por cada order_item vendido
    → buscar receta del menuItem
      → por cada ingredient de la receta con inventoryItemId
        → descontar quantity * orderItem.quantity del stock
        → registrar en stock_movements con movementType='consumo'
        → referencia: numero de orden
```

Modelo de datos — cambios necesarios

Ninguna tabla nueva. Solo mejoras a las existentes.

1. recipe_ingredients — vincular con inventario real

El campo `inventory_item_id` ya existe. Lo que falta es usarlo obligatoriamente cuando el ingrediente corresponde a un ítem del inventario.

No todos los ingredientes necesitan estar en el inventario. Ejemplo: “sal a gusto” no necesita vínculo. Pero “Lomo 200g” sí debe apuntar al ítem “Lomo” de inventario.

2. stock_movements — agregar campos de trazabilidad

```
ALTER TABLE stock_movements
  ADD COLUMN source_type TEXT,
  -- 'restaurant_order' | 'spa_service' | 'manual' | 'purchase' | 'adjustment'
  ADD COLUMN source_id VARCHAR;
  -- ID de la orden, servicio, etc.
```

En `shared/schema.ts`:

```
export const stockMovements = pgTable("stock_movements", {
  // ... campos existentes ...
  sourceType: text("source_type"), // ← nuevo
  sourceId: varchar("source_id"),   // ← nuevo
});
```

FASE 1 — Corregir el bug de stock + vincular recetas al inventario

1.1 Fix del bug de stock en el endpoint

Archivo: `server/routes/inventory.ts`

En el endpoint `POST /api/inventory/movements` , agregar la actualización del ítem después de crear el movimiento:

```
app.post("/api/inventory/movements", requireAuth, async (req, res) => {
  try {
    const { itemId, movementType, quantity, notes, sourceType, sourceId } = req.b

    const item = await storage.getInventoryItem(itemId);
    if (!item) return res.status(404).json({ error: "Artículo no encontrado" });

    const previousStock = item.currentStock ?? 0;
    let newStock = previousStock;

    if (movementType === "entrada") {
      newStock = previousStock + quantity;
    } else if (movementType === "salida" || movementType === "consumo") {
      newStock = previousStock - quantity;
      if (newStock < 0) {
        return res.status(400).json({
          error: `Stock insuficiente para ${item.name}. Stock actual: ${previousS
        });
      }
    } else if (movementType === "ajuste") {
      newStock = quantity;
    }

    const movement = await storage.createStockMovement({
      itemId,
      movementType,
      quantity,
      previousStock,
      newStock,
      notes,
      sourceType: sourceType || "manual",
      sourceId: sourceId || null,
      createdAt: new Date(),
    });
  }
```

```

// ← FIX: actualizar currentStock en el ítem
await storage.updateInventoryItem(itemId, { currentStock: newStock });

// Alerta si baja del mínimo
if (item.minStock && newStock <= item.minStock) {
  console.warn(`[Inventario] Stock bajo: ${item.name} - ${newStock} unidades
}

res.status(201).json({ movement, newStock, item: { ...item, currentStock: new
} catch (error) {
  res.status(500).json({ error: "Error registrando movimiento de stock" });
}
});

```

1.2 Nueva función en storage: deductStockFromOrder

Archivo: server/storage.ts

Esta es la función central de la integración. Se llama al cerrar una orden de restaurant.

```

async deductStockFromOrder(
  orderId: string,
  orderItems: Array<{ menuItemId: string; quantity: number }>
): Promise<{
  deducted: Array<{ itemName: string; quantity: number; unit: string }>;
  warnings: Array<{ itemName: string; required: number; available: number }>;
  skipped: Array<{ ingredientName: string; reason: string }>;
}> {
  const deducted: Array<{ itemName: string; quantity: number; unit: string }> = [
  const warnings: Array<{ itemName: string; required: number; available: number }
  const skipped: Array<{ ingredientName: string; reason: string }> = [];

  for (const orderItem of orderItems) {
    // Buscar receta del ítem del menú
    const recipe = await this.getRecipeByMenuItem(orderItem.menuItemId);
    if (!recipe || recipe.ingredients.length === 0) {
      skipped.push({
        ingredientName: orderItem.menuItemId,
        reason: "Sin receta configurada"
      });
      continue;
    }

    for (const ingredient of recipe.ingredients) {
      // Solo descontar si tiene vínculo con inventario
    }
  }
}

```

```

if (!ingredient.inventoryItemId) {
  skipped.push({
    ingredientName: ingredient.ingredientName,
    reason: "Sin vínculo con inventario"
  });
  continue;
}

const inventoryItem = await this.getInventoryItem(ingredient.inventoryItemI
if (!inventoryItem) {
  skipped.push({
    ingredientName: ingredient.ingredientName,
    reason: "Ítem de inventario no encontrado"
  });
  continue;
}

// Cantidad total a descontar = cantidad de la receta × cantidad vendida
const totalToDeduct = parseFloat(ingredient.quantity) * orderItem.quantity;
const currentStock = inventoryItem.currentStock ?? 0;

// Si no hay stock suficiente: advertir pero NO bloquear el cierre
if (currentStock < totalToDeduct) {
  warnings.push({
    itemName: inventoryItem.name,
    required: totalToDeduct,
    available: currentStock,
  });
  // Descontar lo que hay (llegar a 0, no negativo)
  const toDeduct = Math.min(totalToDeduct, currentStock);
  if (toDeduct <= 0) continue;
}

const actualDeduct = Math.min(totalToDeduct, currentStock);
const newStock = currentStock - actualDeduct;

// Registrar movimiento de stock
await db.insert(stockMovements).values({
  id: randomUUID(),
  itemId: ingredient.inventoryItemId,
  movementType: "consumo",
  quantity: actualDeduct,
  previousStock: currentStock,
  newStock,
  notes: `Consumo automático - Orden ${orderId}`,
  sourceType: "restaurant_order",
  sourceId: orderId,

```

```

        createdAt: new Date(),
    });

    // Actualizar currentStock
    await db.update(inventoryItems)
        .set({ currentStock: newStock })
        .where(eq(inventoryItems.id, ingredient.inventoryItemId));

    deducted.push({
        itemName: inventoryItem.name,
        quantity: actualDeduct,
        unit: inventoryItem.unit,
    });
}

return { deducted, warnings, skipped };
}

```

1.3 Llamar a `deductStockFromOrder` al cerrar una orden

Archivo: `server/routes/restaurant.ts`

En el endpoint `POST /api/restaurant/orders/:id/close` , **después de cerrar la orden exitosamente:**

```

app.post("/api/restaurant/orders/:id/close", async (req, res) => {
    try {
        // ... lógica existente de cierre sin cambios ...

        // Al final, antes del res.json():
        // Descontar stock de inventario
        try {
            const orderItemsList = await storage.getOrderItems(req.params.id);
            const stockResult = await storage.deductStockFromOrder(
                req.params.id,
                orderItemsList.map(i => ({
                    menuItemId: i.menuItemId,
                    quantity: i.quantity
                })))
        };

        // Loguear warnings si hay stock insuficiente
        if (stockResult.warnings.length > 0) {
            console.warn(`[Stock] Advertencias en orden ${req.params.id}:`, stockResu
        }
    }
}

```

```

    // Incluir info de stock en la respuesta
    return res.json({
      ...updatedOrder,
      stockDeducted: stockResult.deducted,
      stockWarnings: stockResult.warnings,
    });
  } catch (stockError) {
    // El descuento de stock NUNCA bloquea el cierre de la orden
    console.error("[Stock] Error en descuento automático:", stockError);
    return res.json(updatedOrder);
  }

  } catch (error) {
    res.status(500).json({ error: "Error cerrando orden" });
  }
});

```

Regla crítica: el descuento de stock está en un try/catch separado. Si falla por cualquier motivo, la orden se cierra igual. La operación comercial nunca se bloquea por un error de inventario.

FASE 2 — Mejorar la UI de recetas para vincular con inventario

2.1 Selector de ítem de inventario en el dialog de recetas

Archivo: client/src/pages/restaurant.tsx

En el dialog de recetas (`isRecipeDialogOpen`), cuando se agrega un ingrediente, mostrar un selector de ítem de inventario además del nombre libre:

```

// Estado adicional para el ingrediente nuevo:
const [newIngredientInventoryId, setNewIngredientInventoryId] = useState<string>(

// Query de ítems de inventario del área restaurant:
const { data: restaurantInventoryItems = [] } = useQuery<InventoryItem[]>({
  queryKey: ["/api/inventory/items", "restaurant"],
  queryFn: () => fetch("/api/inventory/items?area=restaurant").then(r => r.json())
  enabled: isRecipeDialogOpen,
});

```

En el formulario de agregar ingrediente:

```

<div className="space-y-3 border rounded-lg p-3 bg-muted/20">
  <p className="text-sm font-medium">Agregar ingrediente</p>

  {/* Opción 1: Vincular con inventario real */}
  <div className="space-y-1">
    <Label className="text-xs">Artículo del inventario (opcional)</Label>
    <Select
      value={newIngredientInventoryId}
      onChange={(v) => {
        setNewIngredientInventoryId(v);
        if (v) {
          // Autocompletar nombre y unidad desde el inventario
          const item = restaurantInventoryItems.find(i => i.id === v);
          if (item) {
            setNewIngredientName(item.name);
            setNewIngredientUnit(item.unit);
            setNewIngredientCost(item.costPrice || "");
          }
        }
      }}
    >
      <SelectTrigger>
        <SelectValue placeholder="Seleccionar del inventario..." />
      </SelectTrigger>
      <SelectContent>
        <SelectItem value="">Sin vínculo (texto libre)</SelectItem>
        {restaurantInventoryItems.map(item => (
          <SelectItem key={item.id} value={item.id}>
            {item.name} – Stock: {item.currentStock} {item.unit}
          </SelectItem>
        ))}
      </SelectContent>
    </Select>
    {newIngredientInventoryId && (
      <p className="text-xs text-green-600 flex items-center gap-1">
        <CheckCircle className="h-3 w-3" />
        Vinculado – el stock se descontará automáticamente al vender
      </p>
    )}
    {!newIngredientInventoryId && (
      <p className="text-xs text-muted-foreground">
        Sin vínculo – solo para costeo, no descuenta stock
      </p>
    )}
  </div>

```



```

{/* Nombre (autocompletado o libre) */}
<div className="grid grid-cols-3 gap-2">
  <div className="col-span-1">
    <Label className="text-xs">Nombre</Label>
    <Input
      value={newIngredientName}
      onChange={(e) => setNewIngredientName(e.target.value)}
      placeholder="Ej: Lomo"
    />
  </div>
  <div>
    <Label className="text-xs">Cantidad</Label>
    <Input
      type="number"
      value={newIngredientQty}
      onChange={(e) => setNewIngredientQty(e.target.value)}
      placeholder="0.200"
      step="0.001"
    />
  </div>
  <div>
    <Label className="text-xs">Unidad</Label>
    <Select value={newIngredientUnit} onValueChange={setNewIngredientUnit}>
      <SelectTrigger><SelectValue /></SelectTrigger>
      <SelectContent>
        <SelectItem value="g">g</SelectItem>
        <SelectItem value="kg">kg</SelectItem>
        <SelectItem value="ml">ml</SelectItem>
        <SelectItem value="litro">litro</SelectItem>
        <SelectItem value="unidad">unidad</SelectItem>
        <SelectItem value="caja">caja</SelectItem>
      </SelectContent>
    </Select>
  </div>
</div>

<div>
  <Label className="text-xs">Costo unitario ($)</Label>
  <Input
    type="number"
    value={newIngredientCost}
    onChange={(e) => setNewIngredientCost(e.target.value)}
    placeholder="0.00"
    step="0.01"
  />
</div>

```

```

<Button
  size="sm"
  onClick={() => addIngredientMutation.mutate({
    recipeId: currentRecipe!.id,
    inventoryItemId: newIngredientInventoryId || null,
    ingredientName: newIngredientName,
    quantity: newIngredientQty,
    unit: newIngredientUnit,
    unitCost: newIngredientCost || "0",
  })}
  disabled={!newIngredientName || !newIngredientQty}
>
  <Plus className="h-4 w-4 mr-1" /> Agregar
</Button>
</div>

```

2.2 Indicador visual en la lista de ingredientes

En la tabla de ingredientes de la receta, mostrar si está vinculado al inventario:

```

<TableCell>
  {ing.inventoryItemId ? (
    <span className="flex items-center gap-1 text-xs text-green-600">
      <Link className="h-3 w-3" />
      Vinculado
    </span>
  ) : (
    <span className="text-xs text-muted-foreground">Texto libre</span>
  )}
</TableCell>

```

FASE 3 — Depósito y vista de inventario mejorada

3.1 Vista de stock actual por área

Archivo: `client/src/pages/inventory.tsx`

Agregar un selector de área en la vista principal para filtrar ítems:

El endpoint `GET /api/inventory/items` ya acepta `?area=restaurant` . Usarlo con el filtro existente.

Agregar al tab “Artículos” un panel de resumen rápido:

```
// Resumen de stock por estado
const stockCritico = items.filter(i => (i.currentStock ?? 0) <= 0).length;
const stockBajo = items.filter(i =>
  i.minStock && (i.currentStock ?? 0) > 0 && (i.currentStock ?? 0) <= i.minStock
).length;
const stockOk = items.filter(i =>
  !i.minStock || (i.currentStock ?? 0) > i.minStock
).length;

// Panel de 3 cards:
// 🛑 Sin stock: stockCritico artículos
// 🟡 Stock bajo: stockBajo artículos
// 🟢 Stock OK: stockOk artículos
```

3.2 Historial de consumos por orden

En el tab “Movimientos” del inventario, mostrar los movimientos de tipo `consumo` con link a la orden de origen:

```
// En la tabla de movimientos:
{movement.sourceType === "restaurant_order" && (
  <Badge variant="outline" className="text-xs">
    Orden #{movement.sourceId?.slice(-6)}
  </Badge>
)}
```

3.3 Reporte de consumo del período

Nuevo endpoint en `server/routes/inventory.ts`:

```
app.get("/api/inventory/consumo-report", requireAuth, async (req, res) => {
  try {
    const { from, to, area } = req.query;
    const fromDate = from as string || getArgentinaToday();
    const toDate = to as string || getArgentinaToday();

    const consumos = await db.execute(sql`
      SELECT
        ii.id,
        ii.name as item_name,
        ii.unit,
        ii.cost_price,
        SUM(sm.quantity) as total_consumed,
        SUM(sm.quantity * ii.cost_price::numeric) as total_cost,
        COUNT(DISTINCT sm.source_id) as orders_count
    `
```

```

FROM stock_movements sm
JOIN inventory_items ii ON sm.item_id = ii.id
WHERE sm.movement_type = 'consumo'
      AND sm.source_type = 'restaurant_order'
      AND DATE(sm.created_at) BETWEEN ${fromDate} AND ${toDate}
GROUP BY ii.id, ii.name, ii.unit, ii.cost_price
ORDER BY total_cost DESC
`);

const totalCosto = (consumos.rows as any[]).reduce(
  (sum, r) => sum + parseFloat(r.total_cost || "0"), 0
);

res.json({
  from: fromDate,
  to: toDate,
  items: consumos.rows,
  totalCosto,
});
} catch (error) {
  res.status(500).json({ error: "Error generando reporte de consumo" });
}
});

```

En el frontend, agregar un tab “Consumos” en el módulo de inventario que muestre este reporte con selector de período.

FASE 4 — Extensión a otros módulos (después de validar restaurant)

Spa

Los tratamientos del spa pueden tener sus propios insumos (aceites, toallas, productos). Mismo mecanismo que restaurant pero con `spa_treatments` en lugar de `menu_items`.

La tabla `recipeIngredients` no aplica para spa — se necesitaría una tabla `treatment_supplies` análoga:

```

CREATE TABLE treatment_supplies (
  id VARCHAR PRIMARY KEY DEFAULT gen_random_uuid(),
  treatment_id VARCHAR NOT NULL REFERENCES spa_treatments(id),
  inventory_item_id VARCHAR NOT NULL REFERENCES inventory_items(id),

```

```
    quantity DECIMAL(10,3) NOT NULL,  
    unit TEXT NOT NULL  
);
```

Al cerrar un turno de spa, descontar automáticamente los insumos de cada tratamiento realizado.

Orden de implementación recomendado

Semana 1:

- ✓ Fix bug stock (1 línea – updateInventoryItem después de createStockMovement)
- ✓ Agregar sourceType y sourceId a stock_movements (migración simple)
- ✓ Implementar deductStockFromOrder en storage.ts
- ✓ Llamar deductStockFromOrder al cerrar orden de restaurant

Semana 2:

- ✓ Mejorar UI de recetas – selector de ítem de inventario
- ✓ Retroalimentar recetas existentes vinculando con inventario
- ✓ Panel de stock por estado en inventario

Semana 3:

- ✓ Reporte de consumo por período
- ✓ Historial de consumos con referencia a orden

Semana 4+ (después de validar con datos reales):

- ✓ Mapeo cocheras/servicios → inventario
 - ✓ Insumos de spa → inventario
-

Reglas de negocio — IMPORTANTE

1. **El cierre de orden nunca se bloquea por stock insuficiente.** El descuento es automático pero el error de stock solo genera una advertencia, nunca impide la venta.
2. **Los ingredientes sin vínculo a inventario** (texto libre) no generan movimiento de stock. Solo sirven para costeo.
3. **Un ingrediente puede estar en inventario con unidades distintas** a las de la receta. Ejemplo: inventario en kg, receta en g. La conversión de unidades es responsabilidad del operador al cargar la receta (200g en la receta = 0.200 si el inventario está en kg).

4. Los movimientos de consumo quedan auditados con

`sourceType='restaurant_order'` y `sourceId=orderId`, permitiendo trazabilidad completa.

5. Stock negativo no permitido. Si el stock es 0 al momento del cierre, se genera warning pero no se crea movimiento negativo. El sistema descuenta hasta 0 y avisa.

6. Las correcciones de stock se hacen siempre como `ajuste` con motivo obligatorio, nunca editando movimientos anteriores.

Notas técnicas

- `randomUUID` ya está importado en el servidor — no agregar nuevo import
 - `getArgentinaToday()` ya existe en el servidor — reutilizar
 - El endpoint `/api/inventory/items?area=restaurant` ya funciona — no crear nuevo
 - `updateInventoryItem` ya existe en storage — reutilizar
 - Los campos `sourceType` y `sourceId` en `stock_movements` requieren migración con `DEFAULT NULL` — no rompe datos existentes
 - El reporte de consumo usa `sql` template tag de Drizzle — ya disponible
-

MÓDULOS ADICIONALES DE DESCUENTO DE STOCK

Consumos en habitaciones (minibar, amenities)

Cuando recepción carga un cargo tipo minibar o amenity a una habitación, descontar el ítem de inventario correspondiente.

Nueva tabla `charge_inventory_mapping` (Semana 5):

```
CREATE TABLE charge_inventory_mapping (  
  id VARCHAR PRIMARY KEY DEFAULT gen_random_uuid(),  
  description_pattern TEXT NOT NULL,  
  inventory_item_id VARCHAR NOT NULL REFERENCES inventory_items(id),  
  quantity_per_unit DECIMAL(10,3) DEFAULT 1,  
  area TEXT DEFAULT 'general'  
);
```

Permite que cuando se carga "Agua mineral" a una habitación, descuenta 1 botella del inventario de amenities.

Housekeeping (productos de limpieza)

Los insumos de limpieza se descuentan manualmente desde inventario como `salida` con `sourceType='housekeeping'` . En etapa posterior, se puede definir un kit estándar de amenities por tipo de habitación que se descuenta automáticamente al marcar la habitación como limpia.

Orden de implementación corregido

Semana 1 – Base:

- Fix bug stock + `sourceType` en `stock_movements`
- `deductStockFromOrder` en `storage`
- Integrar al cierre de orden de restaurant

Semana 2 – Recetas:

- UI de recetas con selector de inventario real
- Panel de stock por estado

Semana 3 – Reportes:

- Reporte de consumo por período
- Historial con referencia a orden

Semana 4 – Spa:

- Tabla `treatment_supplies`
- Descuento automático al cerrar turno de spa

Semana 5 – Habitaciones y housekeeping:

- Tabla `charge_inventory_mapping`
- Descuento de amenities al cargar cargo en habitación
- Descuento manual housekeeping con `sourceType` correcto